Laboratorio: API serverless observable en GCP

Oct 9, 2026 · @Josh Merida

Práctica en clase: desplegar una API en Cloud Run functions, generar una hora de tráfico con eventos correctos y errores, y analizar sus métricas y logs en un solo dashboard de Cloud Monitoring.

Archivos

```
lab/
  infra/
    versions.tf
    variables.tf
    main.tf
    outputs.tf
    terraform.tfvars
  backend/
    main.py
    requirements.txt
  simulator/
    simulator.py
    config.json
    requests.csv      # lo genera el simulador
```

Backend (`backend/main.py`)

API HTTP en Python con un dominio pequeño a elección (pedidos, pagos, reservaciones) y al menos tres operaciones, por ejemplo crear, consultar y cancelar. La función de entrada se llama `handle`.

- Responde las solicitudes válidas y rechaza las inválidas con 400 (payload inválido), 404 (recurso inexistente) o 409 (cambio de estado imposible).
- Simula fallas propias con dos variables de entorno: `FAILURE_RATE` (probabilidad de responder 500) y `EXTRA_LATENCY_MS` (latencia adicional).

- Escribe una línea JSON en stdout al terminar cada solicitud, con estos campos: `severity`, `message` (`request_completed`), `request_id` (del header `X-Request-ID`), `endpoint`, `status_code`, `outcome` (`success`, `client_error`, `server_error`), `error_type`, `latency_ms` y al menos dos atributos de negocio (por ejemplo `customer_tier`, `category`, `amount`).

Simulador (`simulator/simulator.py`)

Script que corre una hora desde una laptop o VM y hace el papel de los usuarios.

- Genera `n_requests` instantes de envío con distribución normal N(μ, σ) en minutos, truncada a 0 y 60, y dispara cada solicitud en su instante sin esperar a la anterior.
- Mezcla operaciones válidas con una proporción `invalid_share` de errores intencionales (payloads inválidos, IDs inexistentes, secuencias imposibles), y aplica los `scenarios` de la configuración.
- Envía un ID único en `X-Request-ID` y escribe `requests.csv` con una fila por solicitud: `request_id`, instante programado, instante de envío, operación, evento intencionado, código de respuesta y latencia del cliente.

`simulator/config.json`

```
{
  "target_url": "https://<salida function_url de terraform>",
  "n_requests": 6000,
  "mu_min": 30,
  "sigma_min": 10,
  "invalid_share": 0.08,
  "seed": 42,
  "scenarios": [
    { "start_min": 35, "end_min": 45, "effect": "double_errors", "category":
"electronics" }
  ]
}
```

Infraestructura (`infra/`)

Desde `infra/`: `gcloud auth application-default login`, completar `terraform.tfvars`, luego `terraform init` y `terraform apply`. La salida `function_url` va en `config.json`. Si el primer `apply` falla en el build, esperar un minuto y repetirlo. Al

terminar: `terraform destroy`.

`versions.tf`

```
terraform {
  required_version = ">= 1.5"

  required_providers {
    google = {
      source  = "hashicorp/google"
      version = ">= 6.0"
    }
    archive = {
      source  = "hashicorp/archive"
      version = ">= 2.4"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
}
```

`variables.tf`

```
variable "project_id" {
  description = "GCP project ID for this team"
  type        = string
}

variable "region" {
  description = "Region for the function and the source bucket"
  type        = string
  default     = "us-central1"
}

variable "function_name" {
  description = "Function name (lowercase letters, digits and hyphens)"
  type        = string
  default     = "orders-api"
}
```

```
variable "runtime" {
  description = "Cloud Run functions runtime"
  type        = string
  default     = "python313"
}

variable "entry_point" {
  description = "Name of the Python function that handles requests"
  type        = string
  default     = "handle"
}

variable "source_dir" {
  description = "Folder with main.py and requirements.txt"
  type        = string
  default     = "../backend"
}

variable "failure_rate" {
  description = "Probability of a simulated internal failure (0 to 1)"
  type        = number
  default     = 0.02
}

variable "extra_latency_ms" {
  description = "Extra latency added by the backend, in milliseconds"
  type        = number
  default     = 0
}

variable "extra_env" {
  description = "Additional environment variables for the backend"
  type        = map(string)
  default     = {}
}

variable "max_instances" {
  description = "Upper limit for autoscaling"
  type        = number
  default     = 10
}

variable "concurrency" {
```

```
  description = "Concurrent requests per instance"
  type        = number
  default     = 1
}

variable "allow_unauthenticated" {
  description = "Allow public calls so the simulator needs no token"
  type        = bool
  default     = true
}

variable "log_retention_days" {
  description = "Retention for the _Default log bucket"
  type        = number
  default     = 30
}

variable "student_members" {
  description = "IAM members who need access, e.g. user:name@uvg.edu.gt"
  type        = list(string)
  default     = []
}
```

main.tf

```
locals {
  services = [
    "cloudfunctions.googleapis.com",
    "run.googleapis.com",
    "cloudbuild.googleapis.com",
    "artifactregistry.googleapis.com",
    "storage.googleapis.com",
    "logging.googleapis.com",
    "monitoring.googleapis.com",
    "iam.googleapis.com",
  ]

  student_roles = [
    "roles/monitoring.editor",
    "roles/logging.viewer",
    "roles/logging.configWriter",
    "roles/cloudfunctions.viewer",
  ]
```

```
  student_grants = {
    for pair in setproduct(var.student_members, local.student_roles) :
    "${pair[0]}|${pair[1]}" => { member = pair[0], role = pair[1] }
  }
}

# APIs
resource "google_project_service" "apis" {
  for_each           = toset(local.services)
  service            = each.value
  disable_on_destroy = false
}

# Log Analytics on the _Default bucket.
# The upgrade is permanent; destroying this resource does not undo it.
resource "google_logging_project_bucket_config" "default" {
  project          = var.project_id
  location         = "global"
  bucket_id        = "_Default"
  retention_days   = var.log_retention_days
  enable_analytics = true

  depends_on = [google_project_service.apis]
}

# Identities
resource "google_service_account" "runtime" {
  account_id   = "${var.function_name}-run"
  display_name = "Runtime identity for ${var.function_name}"

  depends_on = [google_project_service.apis]
}

resource "google_service_account" "build" {
  account_id   = "${var.function_name}-build"
  display_name = "Build identity for ${var.function_name}"

  depends_on = [google_project_service.apis]
}

resource "google_project_iam_member" "runtime_roles" {
  for_each = toset(["roles/logging.logWriter",
```

```
"roles/monitoring.metricWriter"])
  project  = var.project_id
  role     = each.value
  member   = "serviceAccount:${google_service_account.runtime.email}"
}

resource "google_project_iam_member" "build_roles" {
  for_each = toset(["roles/cloudbuild.builds.builder",
"roles/logging.logWriter"])
  project  = var.project_id
  role     = each.value
  member   = "serviceAccount:${google_service_account.build.email}"
}

resource "google_project_iam_member" "students" {
  for_each = local.student_grants
  project  = var.project_id
  role     = each.value.role
  member   = each.value.member
}

# Source code
resource "google_storage_bucket" "source" {
  name                        = "${var.project_id}-${var.function_name}-src"
  location                    = var.region
  uniform_bucket_level_access = true
  force_destroy               = true

  depends_on = [google_project_service.apis]
}

data "archive_file" "source" {
  type        = "zip"
  source_dir  = var.source_dir
  output_path = "${path.module}/.build/function-source.zip"
}

# The hash in the name forces a redeploy whenever the code changes
resource "google_storage_bucket_object" "source" {
  name   = "function-source-${data.archive_file.source.output_md5}.zip"
  bucket = google_storage_bucket.source.name
  source = data.archive_file.source.output_path
}
```

```
# Function
resource "google_cloudfunctions2_function" "api" {
  name     = var.function_name
  location = var.region

  build_config {
    runtime         = var.runtime
    entry_point     = var.entry_point
    service_account = google_service_account.build.id

    source {
      storage_source {
        bucket = google_storage_bucket.source.name
        object = google_storage_bucket_object.source.name
      }
    }
  }

  service_config {
    available_memory                = "512Mi"
    available_cpu                   = "1"
    timeout_seconds                 = 60
    min_instance_count              = 0
    max_instance_count              = var.max_instances
    max_instance_request_concurrency = var.concurrency
    service_account_email           = google_service_account.runtime.email
    ingress_settings                = "ALLOW_ALL"

    environment_variables = merge(var.extra_env, {
      FAILURE_RATE     = tostring(var.failure_rate)
      EXTRA_LATENCY_MS = tostring(var.extra_latency_ms)
    })
  }

  depends_on = [
    google_project_iam_member.build_roles,
    google_project_iam_member.runtime_roles,
  ]
}

# Public access for the simulator
resource "google_cloud_run_v2_service_iam_member" "public" {
```

```
  count    = var.allow_unauthenticated ? 1 : 0
  project  = var.project_id
  location = var.region
  name     = google_cloudfunctions2_function.api.name
  role     = "roles/run.invoker"
  member   = "allUsers"
}
```

outputs.tf

```
output "function_url" {
  description = "URL for the simulator"
  value       = google_cloudfunctions2_function.api.url
}

output "log_analytics_view" {
  description = "Table name for SQL queries in Log Analytics"
  value       = "`${var.project_id}.global._Default._Default`"
}

output "dashboards_console" {
  value = "https://console.cloud.google.com/monitoring/dashboards?
project=${var.project_id}"
}

output "log_analytics_console" {
  value = "https://console.cloud.google.com/logs/analytics?
project=${var.project_id}"
}
```

terraform.tfvars

```
project_id      = "my-team-project"
region          = "us-central1"
function_name   = "orders-api"
failure_rate    = 0.02
student_members = []
```

Dashboard

Cada equipo define qué preguntas quiere responder y arma un solo dashboard de Cloud Monitoring con métricas nativas de la función, métricas basadas en logs, gráficas SQL de Log Analytics y un panel de logs. Las métricas basadas en logs se crean antes de correr el simulador, porque no cuentan logs anteriores.

Fuentes

- Functions Framework for Python
- Structured logging
- HTTPX: Async Support y numpy Generator.normal
- Terraform: Get Started on Google Cloud
- Log-based metrics overview
- Query and analyze logs with Log Analytics y Chart SQL query results
- Create and manage custom dashboards y Display logs on a dashboard